

Unity · Puzzle Platformer

Collectables & Level Config System

Robot Parts · Memory Shards · Per-Level Data

What was built · How to implement · How it works

Technical documentation

Branch: `gameplay/mechanism`

Generated 16 July 2026 · Revision 2

1. What was built

Two systems that work together:

- **Collectables** — persistent pickups (**Robot Parts** and **Memory Shards**) the player gathers by walking into them; once collected they never reappear.
- **Per-level configuration** — one **ScriptableObject per level** holding *all* that level's tunable data (collectable counts, camera dead-zone, sequence settings, room for more). Components read their values from it instead of exposing their own Inspector fields.

Collectable	HUD	Rule
Robot Parts	0/56	One fixed game-wide total. Parts placed per level are recorded in each level's config.
Memory Shards	0/6 , 0/20 , ...	Grouped into story tiers by level range (levels 1–12 need 6 → Story 1; 13–50 need 20 → Story 2). Fully data-driven, not hard-coded.

Key behaviours

- **Permanent** — collected pickups never reappear, across replays and app restarts.
- **One config asset per level** — a level's camera, sequence and collectable counts live in a single `LevelConfig`.
- **One manager per scene** — a single `LevelManager` object hosts both the `CollectableLevelManager` and the `LevelContext` that holds the config.
- **Editable rules** — the Robot Parts total and the shard story tiers live in one constants file.
- **Tooling** — editor windows create prefabs & per-level configs, build the HUD, inject everything into all scenes (migrating existing values), and reset saved data.
- **New Game resets** — starting a new game currently wipes all collectable progress.

Files

```
Assets/MainGame/Scripts/Level/
├─ LevelConfig.cs      ★ ScriptableObject, one per level (all level data)
├─ LevelContext.cs    per-scene holder of the LevelConfig (runs first)

Assets/MainGame/Scripts/Collectables/
├─ CollectableType.cs    enum: RobotPart | MemoryShard
├─ CollectableConstants.cs ★ editable dials: 56 total + shard story tiers
├─ CollectableSaveData.cs serialisable save DTO
├─ CollectableSaveSystem.cs JSON persistence (persistentDataPath)
├─ CollectableLevelManager.cs per-scene coordinator (RequireComponent LevelContext)
├─ Collectable.cs        the pickup component
├─ CollectableHUD.cs     on-screen counters
├─ README.md
├─ Editor/
│   ├─ CollectableToolsWindow.cs    reset · prefabs · create LevelConfig · ids
│   ├─ CollectableUISetupWindow.cs  build HUD · inject LevelManager + configs
│   └─ CollectableToolsPaths.cs     shared helper
```

Assets/MainGame/Prefabs/Collectables/ CollectableHUD, RobotPart, MemoryShard
Assets/MainGame/ScriptableObjects/LevelConfigs/ Level1Config, Level2Config, ...

Changed from revision 1: the single `CollectableDatabase` was replaced by one `LevelConfig` per level. Camera and sequence settings moved into that config, and the two manager components were merged onto a single `LevelManager` object.

2. How it works

2.1 Architecture at a glance

Layer	Type	Responsibility
Config (rules)	<code>CollectableConstants</code>	Robot Parts grand total; Memory Shard story tiers.
Config (per level)	<code>LevelConfig</code> (SO)	Collectable counts, camera dead-zone, sequence settings — one asset per level.
Scene holder	<code>LevelContext</code>	Holds the level's config. Runs first; exposes <code>Config</code> and <code>CurrentLevel</code> .
Persistence	<code>CollectableSaveSystem</code>	Reads/writes <code>collectables.json</code> ; tracks collected ids & counts.
Coordinator	<code>CollectableLevelManager</code>	Single access point for collect + queries; reads level number from <code>LevelContext</code> .
World object	<code>Collectable</code>	The pickup. Detects the player, records itself, hides.
UI	<code>CollectableHUD</code>	Shows the two counters; refreshes on collect.

Single assignment point. Each scene has one `LevelManager` object carrying both `CollectableLevelManager` and `LevelContext`. You assign the level's `LevelConfig` to the `LevelContext` once; camera, sequence, collectables and HUD all read from it.

2.2 The `LevelConfig` asset

```
LevelConfig (ScriptableObject)
├── levelNumber           // drives the level number everywhere
├── robotPartCount, memoryShardCount // authoring: how many placed in this level
├── cameraDeadZone       // deadZoneX/Y, offset, smoothTime,
                        // minX/maxX/minY/maxY, followX/followY
└── sequence             // maxSequenceLength, requireFullSequence
```

`CameraFollowDeadZone` and `SequenceManager` no longer show these fields in the Inspector. On `Awake` they copy the values from `LevelContext.Instance.Config`. The old serialized fields are kept but hidden, purely as a fallback and to support one-time migration.

2.3 Level identity

The level number comes from the config's `levelNumber` (via `LevelContext.CurrentLevel`), falling back to the scene build index (Home = 0, Level1 = 1, ...). `CollectableLevelManager` reads it from the `LevelContext` on its own object.

2.4 Persistence & the "never respawn" rule

Each placed pickup carries a stable unique id (a GUID per scene instance). When collected, a record is written to the save file:

```
{
  "records": [
    { "id": "a1b2c3...", "type": 0, "level": 1 },    // RobotPart in level 1
    { "id": "d4e5f6...", "type": 1, "level": 2 }    // MemoryShard in level 2
  ]
}
```

File: `Application.persistentDataPath/collectables.json`. On load each `Collectable` asks whether its id is already saved — if so it deactivates in `Start()`, so it is never seen again. Recording is idempotent, so replaying a level cannot double-count.

2.5 Counting

Robot Parts HUD	<code>totalCollected / RobotPartsGrandTotal</code> — a single game-wide total (the <code>56</code> constant).
Memory Shards HUD	<code>collectedInTier / tierRequired</code> — progress within the current level's tier. A level outside every tier hides the shard widget.

2.6 Collect flow

```
Player enters trigger
→ Collectable.OnTriggerEnter2D (tag == "Player")
→ CollectableLevelManager.Collect(id, type)
  → CollectableSaveSystem.MarkCollected(id, type, currentLevel)    // writes JSON
  → OnCollectableCollected event fired
→ AudioManager.PlayPickup(); optional VFX; object deactivates
CollectableHUD (subscribed) → Refresh() → labels update
```

2.7 Initialisation order

`LevelContext` is marked `[DefaultExecutionOrder(-1000)]` so it initialises before everything else. Camera, sequence and the collectable manager can therefore read the config safely in their own `Awake`.

2.8 Story unlocks (future store)

```
CollectableLevelManager.Instance.IsCurrentStoryUnlocked;    // this level's tier
CollectableLevelManager.IsStoryUnlockedForLevel(level);      // any tier, from anywhere
```

3. How to implement (setup)

Step 1 — Prefabs & HUD (one-time)

- `Tools ▶ Collectables ▶ Collectable Tools` → **Create Robot Part Prefab / Create Memory Shard Prefab**. Assign a sprite to each; leave `Unique Id` empty.
- `Tools ▶ Collectables ▶ Setup Collectable UI` → (optional) assign icons → **Build / Update HUD Prefab**.

Step 2 — Inject per-level setup (automated + migration)

In `Setup Collectable UI`, keep *Ensure Level Manager* and *Create + assign LevelConfig* on, then **Setup In All Build Scenes** (or **Setup Current Scene** for scenes not in Build Settings). Per scene it:

- adds the HUD prefab (if missing);
- creates a single `LevelManager` object holding both `CollectableLevelManager` and `LevelContext` — consolidating any separate objects an earlier setup made;
- creates `ScriptableObjects/LevelConfigs/Level{n}Config.asset` and **captures that scene's current camera dead-zone + sequence values into it**, then assigns it to the `LevelContext` ;
- saves the scene.

Idempotent — safe to re-run. *Only 'Level*' Scenes* skips Home/menu.

Nothing is lost. Because the tool reads each scene's *existing* camera bounds and sequence values and writes them into the new config, per-level tuning survives the move into the asset.

Step 3 — Fill in & place content

- Open each `Level{n}Config` and set **Level Number**, **Robot Part Count**, **Memory Shard Count**, and adjust camera / sequence if needed.
- Drop the Robot Part / Memory Shard prefabs into the level (each self-assigns a unique id; if any is blank use *Collectable Tools ▶ Assign Missing Ids*).
- Ensure the pickup's collider is a **trigger** and the player is tagged `Player` .

Creating a config by hand

`Assets ▶ Create ▶ Level ▶ Level Config`, or `Collectable Tools ▶ Create Level Config` (enter a level number). Assign it to that scene's `LevelManager → Level Context → Config` .

Tuning

Change	Where
A level's camera / sequence / collectable counts	that level's <code>LevelConfig</code> asset
Robot Parts grand total (56)	<code>CollectableConstants.RobotPartsGrandTotal</code>

Add / edit a Memory Shard story tier	<code>CollectableConstants.MemoryShardTiers</code> (no overlaps)
HUD position / colours / canvas order	the <code>CollectableHUD.prefab</code>

```
// CollectableConstants.cs - the story tiers you edit
public static readonly MemoryShardTier[] MemoryShardTiers =
{
    new MemoryShardTier(fromLevel: 1, toLevel: 12, required: 6, storyId: "Story_1"),
    new MemoryShardTier(fromLevel: 13, toLevel: 50, required: 20, storyId: "Story_2"),
};
```

Resetting data

- **Manually:** `Collectable Tools ▶ Reset Collectable Data` (deletes the save file after a confirmation).
- **Automatically:** the **New Game** button (`MenuManager.NewGame()`) calls `CollectableSaveSystem.ResetAll()` . Remove that line later when collectables should persist.

Quick reference — public API

Member	Returns
<code>LevelContext.Instance.Config</code>	The scene's LevelConfig.
<code>LevelContext.Instance.CurrentLevel</code>	Level number (config, else build index).
<code>CollectableLevelManager.Instance</code>	The scene's collectable coordinator.
<code>.RobotPartsCollectedTotal</code> / <code>.RobotPartsGrandTotal</code>	Parts collected game-wide / fixed total (56).
<code>.MemoryShardsCollectedInTier</code> / <code>.MemoryShardsTierTarget</code>	Shard progress for the current tier.
<code>.RobotPartCountThisLevel</code> / <code>.MemoryShardCountThisLevel</code>	Authoring counts from the level config.
<code>.IsCurrentStoryUnlocked</code>	Current tier complete?
<code>CollectableLevelManager.IsStoryUnlockedForLevel(int)</code>	Any tier complete? (static)
<code>CollectableSaveSystem.ResetAll()</code>	Wipe all progress.